

Multi-Precision Arithmetic Calculator (Large Numbers)

Computer Organization and Assembly Language Lab Project

Project Report

Group Members and Modules

Member	Name	Module
Member 1	Aleeza Noor	Input, Validation, Storage Module and Expression Parsing Support
Member 2	Hina Naeem	Addition, Subtraction, Comparison Module and Expression Evaluation Support
Member 3	Ayesha Khan	Multiplication Module and Advanced Expression Operations
Member 4	Urwa Abbasi	Main Menu, Output, Analysis, Integration Module and Expression Mode Coordination

1. Introduction

The Multi-Precision Arithmetic Calculator is an Assembly Language console application designed to perform arithmetic operations on numbers larger than the normal 32-bit register limit. The project implements large-number addition, subtraction, and multiplication using arrays of digits. It follows a modular development structure so that each member contributes a separate technical component and also participates in testing and integration.

The project was later extended with an Expression Mode module (`expression.asm`), which adds a full expression evaluator that supports BODMAS/operator precedence, multi-bracket types, power, factorial, modulo, and multiple number systems (binary, octal, hexadecimal) directly inside user-entered expressions. To keep the collaborative structure of the project consistent, the procedures inside `expression.asm` were divided equally among all four group members, with each member taking ownership of the procedures that best matched their original area of responsibility.

2. Problem Statement

Standard integer registers cannot directly store very large numbers such as 50-digit or 100-digit values. The problem is to design a calculator that accepts large numeric strings from the user, validates the input, stores the digits in memory, performs arithmetic manually digit by digit, and displays the correct result. The program must also handle negative numbers and invalid input.

Additionally, the extended expression mode solves the problem of evaluating complex arithmetic expressions that include operator precedence, brackets, powers, factorials, and multiple number system prefixes — all within a single user-entered string.

3. Objectives

The main objectives are:

- Read large numbers from user input.
- Validate numeric input and reject invalid characters.
- Store numbers as reversed digit arrays.
- Support positive and negative signs.
- Perform large-number addition, subtraction, and multiplication.
- Display results clearly through a menu-driven console interface.
- Provide analysis related to digit counts, memory usage, and system behavior.
- Evaluate full expressions with BODMAS precedence, brackets, power, factorial, and modulo.
- Support binary (0b), octal (0o), and hexadecimal (0x) number system prefixes inside expressions.
- Use GitHub collaboration with visible member contributions.

4. Tools and Libraries

The project was developed using Microsoft Visual Studio 2022, MASM, and the Irvine32 library. GitHub and GitHub Desktop were used for version control and collaboration.

5. System Design

The program is divided into multiple Assembly source files. The main module controls the menu and user flow. The input module reads user input, validation checks correctness, and storage converts valid strings into reversed digit arrays. Arithmetic modules process the arrays and store results in a shared result array. The output module prints the final result, while the analysis module displays digit counts and fixed memory usage.

The expression module (expression.asm) integrates directly with the main menu as option 4. It implements a recursive descent parser that evaluates an entire expression string in a single pass. It reuses the shared result, resultLen, and resultSign variables and calls DisplayResult and ClearArray from other modules.

Data Flow (Standard Modes)

User input → ReadNumber → ValidateNumber → StringToArray → Arithmetic module → result array → DisplayResult → RunAnalysis

Data Flow (Expression Mode)

User input → RunExpressionMode → ParseExpression → ParseTerm → ParsePower → ParseFactor → ParsePrimary → ParseNumber → SaveSignedResult → DisplayResult → RunAnalysis

6. Module Distribution and Team Contribution

Each member led one core module of the calculator and additionally contributed an equal share of the Expression Mode module (expression.asm). The expression.asm procedures were divided into four groups of approximately equal size and complexity, with each group assigned to the member whose responsibilities most closely matched the function of those procedures.

6.1 Member 1 — Aleeza Noor (Input, Validation, and Storage Module)

Member 1 led the input and storage module. This included reading large numbers, checking invalid characters, handling signs, and converting input strings into reversed digit arrays. Member 1 also helped test input-output behavior during integration.

Expression Mode contribution: Member 1 implemented the low-level character reading and number-parsing procedures of expression.asm — SkipSpaces, PeekChar, GetChar, and ParseNumber. These procedures form the input layer of the expression evaluator: they read raw characters from the expression buffer, skip whitespace, and convert decimal, binary, octal, and hexadecimal digit sequences into 32-bit values. This mirrors Member 1's role in the core calculator, where input is read and converted into a usable internal format.

6.2 Member 2 — Hina Naeem (Addition, Subtraction, and Comparison Module)

Member 2 led the arithmetic core for addition and subtraction. This included comparing large numbers, handling carry and borrow, and determining result signs. Member 2 also helped ensure arithmetic modules followed the shared storage format.

Expression Mode contribution: Member 2 implemented the additive and multiplicative evaluation layers — ParseExpression and ParseTerm — along with SaveSignedResult. ParseExpression evaluates left-to-right addition and subtraction, while ParseTerm evaluates multiplication, division, and modulo, including the division-by-zero check. SaveSignedResult converts the final signed 32-bit value back into the shared reversed-digit result array and sign flag, the same format Member 2's addition and subtraction module produces in core calculator mode.

6.3 Member 3 — Ayesha Khan (Multiplication Module)

Member 3 led multiplication. The multiplication module uses long multiplication on reversed arrays and stores the product in the shared result array. It handles carry propagation, negative signs, and zero inputs.

Expression Mode contribution: Member 3 implemented the repeated-multiplication procedures — ParsePower and ParseFactor. ParsePower evaluates the right-associative power operator using a multiplication loop, and ParseFactor evaluates the postfix factorial operator using a similar repeated-multiplication loop with a negative-value check. Both procedures rely on the same imul-based repetition logic that underlies Member 3's core multiplication module.

6.4 Member 4 — Urwa Abbasi (Main Menu, Output, Analysis, and Integration Module)

Member 4 led the main menu, output formatting, analysis display, and final integration of all modules.

Expression Mode contribution: Member 4 implemented the entry point and structural/error-handling procedures — RunExpressionMode, ParsePrimary, MarkBaseError, and MarkCharError. RunExpressionMode is the option-4 menu entry point that reads the expression string, calls the parser, and displays the correct result or error message. ParsePrimary handles numbers, all three bracket types, and unary plus/minus, tying the whole recursive descent chain together. MarkBaseError and MarkCharError set the shared error flags used by RunExpressionMode to choose the correct error message. This matches Member 4's role of integrating all procedures into a single working menu option.

6.5 Summary of expression.asm Distribution

Member	Procedures Assigned	Count
Member 1 — Aleeza Noor	SkipSpaces, PeekChar, GetChar, ParseNumber	4
Member 2 — Hina Naeem	ParseExpression, ParseTerm, SaveSignedResult	3
Member 3 — Ayesha Khan	ParsePower, ParseFactor	2
Member 4 — Urwa Abbasi	RunExpressionMode, ParsePrimary, MarkBaseError, MarkCharError	4

This distribution gives each member three to four procedures, keeping the workload balanced while keeping each member's expression.asm portion conceptually linked to the module they already owned in the core calculator.

7. Implementation Details

The project stores each large number in a byte array where each byte contains one digit from 0 to 9. Digits are stored in reverse order so that index 0 contains the least significant digit. This makes arithmetic easier because addition, subtraction, and multiplication all begin from the rightmost digit.

The sign of each number is stored separately. A value of 0 means positive and a value of 1 means negative. This avoids mixing sign characters with digit arrays. The result uses the same format: result array, result length, and result sign.

Important Shared Variables

- num1, num2 — reversed digit arrays (200 bytes each)

- result — reversed result array (400 bytes)
- num1Len, num2Len, resultLen — digit counts
- num1Sign, num2Sign, resultSign — sign flags

Expression Module Internal Variables (expression.asm)

The expression module uses its own local variables for parsing state:

- exprBuffer — stores the raw input string (up to 120 characters)
- exprPos — pointer tracking current parse position inside exprBuffer
- exprError — set to 1 if any error is detected during parsing
- baseError — set if an invalid number base prefix or digit is found
- charError — set if an unsupported character is encountered
- divZeroError — set if division or modulo by zero is attempted
- factError — set if factorial is applied to a negative number

8. Expression Mode Module (expression.asm)

The expression.asm module adds a complete recursive descent expression evaluator to the calculator. It is called from main.asm when the user selects option 4 in the menu. The module handles full BODMAS operator precedence, bracket types, right-associative power, factorial, modulo, and multiple number system prefixes. As described in Section 6, its thirteen procedures are divided equally among the four group members.

8.1 Operator Precedence Hierarchy

Level	Procedure	Operators / Handled
Lowest (1)	ParseExpression	+ and - (left-associative)
2	ParseTerm	*, /, % (left-associative)
3	ParsePower	^ power (right-associative)
4	ParseFactor	! factorial (postfix)
Highest (5)	ParsePrimary / ParseNumber	Numbers, brackets, unary +/-

8.2 Supported Number Systems

ParseNumber handles four number system formats:

Prefix	Base	Example	Invalid Form
None	Decimal (10)	25, 100	—
0b or 0B	Binary (2)	0b1010	0/b1010
0o or 0O	Octal (8)	0o17	0/o17
0x or 0X	Hexadecimal (16)	0xA, 0x1F	0/xA

8.3 Bracket Types

ParsePrimary supports three interchangeable bracket types that can be nested:

- Round brackets: (expression)
- Square brackets: [expression]
- Curly brackets: { expression }

Mismatched brackets (such as [2 + 3)) trigger an invalid expression error.

8.4 Procedures in expression.asm, Grouped by Member

The table below lists every procedure in expression.asm, its purpose, and the member responsible for it.

Procedure	Purpose	Owner
SkipSpaces	Advances parse position past whitespace	Aleeza Noor (M1)
PeekChar	Reads next non-space character without consuming it	Aleeza Noor (M1)
GetChar	Reads and consumes next non-space character	Aleeza Noor (M1)
ParseNumber	Parses decimal, binary (0b), octal (0o), hex (0x)	Aleeza Noor (M1)
ParseExpression	Handles + and - with left-to-right evaluation	Hina Naeem (M2)
ParseTerm	Handles *, /, % with left-to-right evaluation	Hina Naeem (M2)
SaveSignedResult	Converts signed EAX into the shared reversed digit result array	Hina Naeem (M2)
ParsePower	Handles ^ with right-to-left (right-associative) evaluation	Ayesha Khan (M3)
ParseFactor	Handles postfix ! factorial	Ayesha Khan (M3)
RunExpressionMode	Entry point — reads input, calls parser, reports result or error	Urwa Abbasi (M4)
ParsePrimary	Handles numbers, brackets, unary + and -	Urwa Abbasi (M4)
MarkBaseError	Sets base error flags for invalid prefixes or out-of-range digits	Urwa Abbasi (M4)
MarkCharError	Sets char error flag for unsupported characters	Urwa Abbasi (M4)

8.5 Error Handling in Expression Mode

Error Type	Condition	Message Displayed
Invalid expression	Mismatched brackets or malformed syntax	Invalid expression! Check operators and brackets.
Base error	Wrong prefix (0/b) or digit out of range for base	Invalid number system! Use 0b1010, 0o7, or 0xA. Do not write 0/b or 0/a.
Char error	Unsupported characters in expression	Invalid character! Only digits, A-F for hex, operators, and brackets are allowed.
Division by zero	Divisor or modulo operand is 0	Division by zero is not allowed!
Factorial error	Factorial applied to negative number	Factorial only works for zero or positive values!

9. Addition and Subtraction Logic

Addition works digit by digit with carry. If both numbers have the same sign, their magnitudes are added and the result keeps the same sign. If the signs are different, the program compares magnitudes and subtracts the smaller magnitude from the larger one.

Subtraction is implemented by flipping the sign of the second number and reusing the addition logic. This keeps the code simpler and reduces repeated sign-handling logic.

10. Multiplication Logic

Multiplication uses the standard long multiplication method. Every digit of the first number is multiplied by every digit of the second number. The partial products are added at index $i + j$ because the arrays are reversed. Carry is propagated after each digit multiplication. The final result is trimmed to remove unnecessary leading zeroes.

11. Integration Strategy

Each module was first tested independently using temporary test main files. These temporary files were not kept as final project source files. After module testing, the final main menu was connected with all shared procedures. The integrated project was then rebuilt and tested through Visual Studio.

Only one final file contains main PROC: main.asm. Other files expose procedures using PUBLIC and access shared variables using EXTERN. This avoids duplicate entry points and linker errors.

The expression.asm module integrates with main.asm through a single PUBLIC entry point, RunExpressionMode. It accesses the shared result, resultLen, and resultSign variables via EXTERN declarations. This keeps the module self-contained and consistent with the overall design, while still allowing each member's assigned procedures within expression.asm to be developed and tested independently before integration.

12. Testing and Debugging Strategy

The project was tested using valid large numbers, negative numbers, zero cases, carry-heavy cases, and invalid input. Linker errors during development were resolved by matching procedure declarations and using PROTO/PUBLIC/EXTERN consistently. Duplicate test main files were excluded from the final build.

12.1 Core Arithmetic Test Cases

Test	Input	Expected Result
Addition	999999 + 888888888888	888889888887
Subtraction	10000000000000000000 - 999999999999999999	1
Negative subtraction	123456789 - 987654321	-864197532
Multiplication	12345678901234567890 × 9876543210	121932631124828532111263526900
Negative multiplication	9999999000 × -333	-3329999667000
Zero multiplication	0 × 98765432109876543210	0
Invalid input	999999999aaa	Rejected with error message

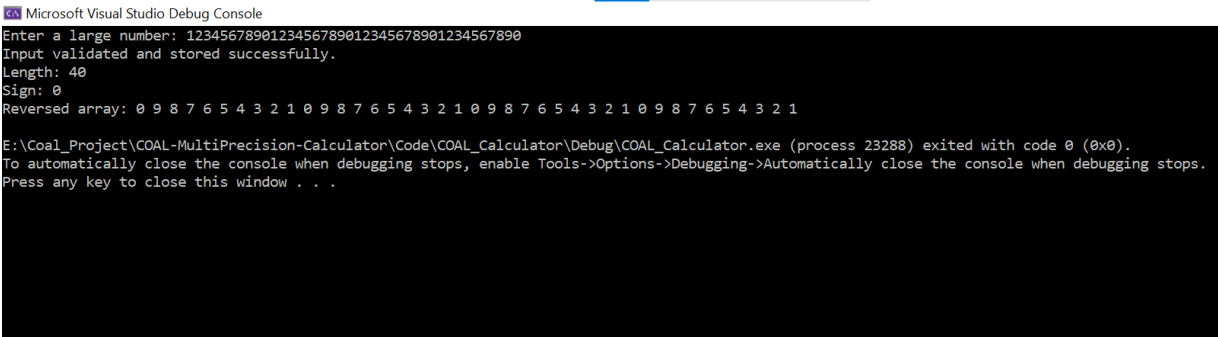
12.2 Expression Mode Test Cases

Test	Expression	Expected Result
Basic BODMAS	$2 + 3 * 4$	14
Brackets override BODMAS	$(2 + 3) * 4$	20
Unary negative	$-(2 + 3) * 4$	-20
All number systems	$0b1010 + 0o12 + 0xA + 10$	40
Nested brackets	$[2^{(3+1)} + \{5! / (0b101 + 5)\}] - 0xA$	18
All features combined	$\{[0b1010 + (0o7 * 2)] + 0xA\} \% 6 + 2^3 + 3!$	18
Right-associative power	2^3^2	512
Division by zero	$5 / 0$	Division by zero is not allowed!

Invalid base prefix	0b102 + 5	Invalid number system error
Wrong brackets	[2 + 3)	Invalid expression error

12.3 Input Validation Test Cases (Member 1)

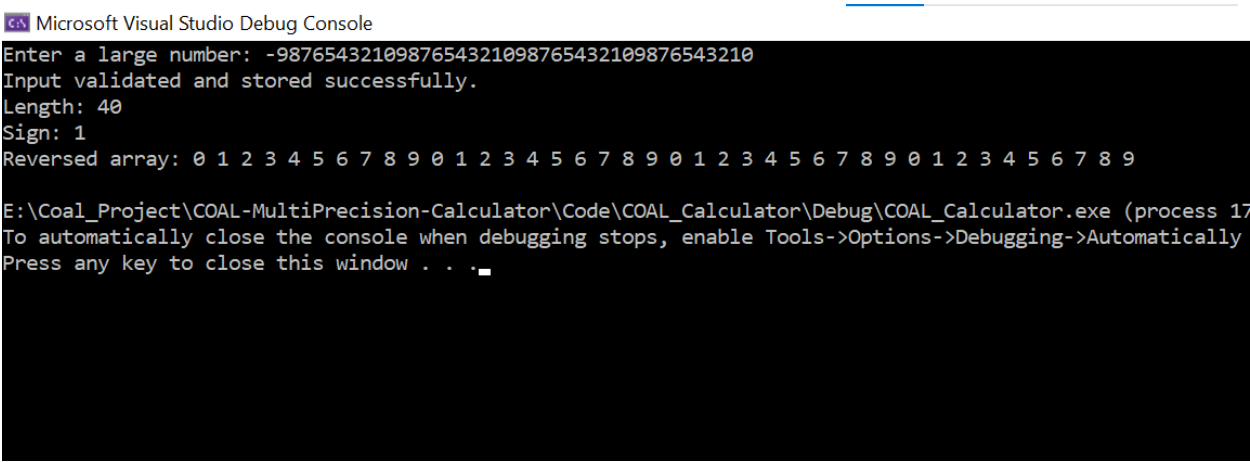
The following figures document the input validation tests run on the storage module:



```
Microsoft Visual Studio Debug Console
Enter a large number: 1234567890123456789012345678901234567890
Input validated and stored successfully.
Length: 40
Sign: 0
Reversed array: 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe (process 23288) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Figure 1: Positive large number — input validated and stored successfully, showing reversed digit array



```
Microsoft Visual Studio Debug Console
Enter a large number: -9876543210987654321098765432109876543210
Input validated and stored successfully.
Length: 40
Sign: 1
Reversed array: 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe (process 17
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically
Press any key to close this window . . .
```

Figure 2: Negative large number — stored with sign flag = 1 and reversed digit array

```
Microsoft Visual Studio Debug Console
Enter a large number: 12345678901234567890A12345678901234567890
Invalid input! Try again.

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Aut
Press any key to close this window . . .
```

Figure 3: Invalid character test — alphabetic character A in input rejected

```
Microsoft Visual Studio Debug Console
Enter a large number: 98765432109876543210#98765432109876543210
Invalid input! Try again.

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Aut
Press any key to close this window . . .
```

Figure 4: Invalid symbol test — # symbol in input rejected with error message

12.4 Expression Mode Test Figures

```
E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe
-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets ( ) [ ] { }, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: 2 + 3 * 4
Result: 14
```

Figure 5: Basic BODMAS — $2 + 3 * 4 = 14$ (multiplication evaluated before addition)

```

-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: (2 + 3) * 4
Result: 20

```

Figure 6: Brackets override BODMAS — $(2 + 3) * 4 = 20$

```

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe
-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: -(2 + 3) * 4
Result: -20

Analysis Summary
Input 1 digits: 0
Input 2 digits: 0
Result digits: 2
Static digit storage used: num1=200 bytes, num2=200 bytes, result=400 bytes
Large-number arithmetic is performed digit-by-digit on reversed arrays.

```

Figure 7: Unary negative — $-(2 + 3) * 4 = -20$

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe

```
-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: 0b1010 + 0o12 + 0xA + 10
Result: 40

Analysis Summary
Input 1 digits: 0
Input 2 digits: 0
Result digits: 2
Static digit storage used: num1=200 bytes, num2=200 bytes, result=400 bytes
Large-number arithmetic is performed digit-by-digit on reversed arrays.
```

Figure 8: All number systems — $0b1010 + 0o12 + 0xA + 10 = 40$

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe

```
-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: [2^(3+1) + {5! / (0b101 + 5)}] - 0xA
Result: 18

Analysis Summary
Input 1 digits: 0
Input 2 digits: 0
Result digits: 2
Static digit storage used: num1=200 bytes, num2=200 bytes, result=400 bytes
Large-number arithmetic is performed digit-by-digit on reversed arrays.
```

Figure 9: Nested brackets — $[2^{(3+1)} + \{5! / (0b101 + 5)\}] - 0xA = 18$

```

-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: {[0b1010 + (0o7 * 2)] + 0xA} % 6 + 2^3 + 3!
Result: 18

Analysis Summary
Input 1 digits: 0
Input 2 digits: 0
Result digits: 2
Static digit storage used: num1=200 bytes, num2=200 bytes, result=400 bytes
Large-number arithmetic is performed digit-by-digit on reversed arrays.

```

Figure 10: Full expression — all bracket types, bases, modulo, power, and factorial combined = 18

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe

```

-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: 2^3^2
Result: 512

Analysis Summary
Input 1 digits: 0
Input 2 digits: 0
Result digits: 3
Static digit storage used: num1=200 bytes, num2=200 bytes, result=400 bytes
Large-number arithmetic is performed digit-by-digit on reversed arrays.

```

Figure 11: Right-associative power — $2^3^2 = 512$ (evaluated as $2^{(3^2)} = 2^9$)

```

-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: 5 / 0
Division by zero is not allowed!

```

Figure 12: Division by zero — 5 / 0 produces correct error message

```

-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: 0b102 + 5
Invalid number system! Use 0b1010, 0o7, or 0xA. Do not write 0/b or 0/a.

```

Figure 13: Invalid base prefix — 0b102 rejected with specific error message

E:\Coal_Project\COAL-MultiPrecision-Calculator\Code\COAL_Calculator\Debug\COAL_Calculator.exe

```

-----
Multi-Precision Arithmetic Calculator
1. Addition
2. Subtraction
3. Multiplication
4. Expression Mode - choose this before typing full expression
5. Exit
Enter choice: 4

Use + - * / % ^ ! brackets () [] {}, and prefixes 0b 0o 0x
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.
Enter expression: [2 + 3)
Invalid expression! Check operators and brackets.

```

Figure 14: Wrong brackets — [2 + 3) rejects with invalid expression error

12.5 Output Display

Figure 15: Main menu output display showing menu options and expression mode entry

```
-----  
Multi-Precision Arithmetic Calculator  
1. Addition  
2. Subtraction  
3. Multiplication  
4. Expression Mode - choose this before typing full expression  
5. Exit  
Enter choice: 4  
  
Use + - * / % ^ ! brackets ( ) [ ] { }, and prefixes 0b 0o 0x  
Examples: 0b1010, 0o7, 0xA. Do not use 0/b, 0/o, or 0/a.  
Enter expression:
```

13. Analysis and Evaluation

The maximum input size is 200 digits per number. The first number uses 200 bytes, the second number uses 200 bytes, and the result array uses 400 bytes. Addition and subtraction have $O(n)$ time complexity. Multiplication has $O(n \times m)$ time complexity because it compares each digit of one input with each digit of the other input.

The expression mode evaluates expressions up to 120 characters. All arithmetic inside the expression evaluator uses 32-bit signed EAX registers, so intermediate results are limited to 32-bit range. The final result is converted into the shared reversed digit array before display, meaning the output format is consistent with all other modes.

The program is efficient for the project scope because all storage is fixed and predictable. The reversed array format reduces complexity in arithmetic operations.

14. GitHub Collaboration Workflow

The group used GitHub for collaboration. Members committed their own module files and screenshots. Final integration was tested after all modules were available. During integration, some placeholder files and duplicate copies were cleaned so that the final Code folder contains one clear set of source files.

For expression.asm specifically, each member developed and tested their assigned procedures (as listed in Section 8.4) before committing them, allowing Member 4 to merge all four contributions into a single integrated source file without conflicts.

15. Challenges Faced

The main challenges were managing procedure names between modules, avoiding duplicate main PROC definitions, handling GitHub account/commit identity correctly, and keeping Visual Studio build settings consistent. Another challenge was ensuring old placeholder files did not overwrite final module code during integration.

For the expression module, additional challenges included correctly implementing right-associativity for the power operator, ensuring unary minus is handled recursively in ParsePrimary, and making sure mismatched bracket types are detected at parse time. Because the module's procedures were split across all four members, an additional challenge was agreeing on the shared error flags (exprError, baseError, charError, divZeroError, factError) and the calling convention (PeekChar/GetChar) early on, so that each member's procedures would interoperate correctly once merged. These issues were resolved through careful testing of individual parse procedures before full integration.

16. Conclusion

The final project successfully implements a modular multi-precision arithmetic calculator in Assembly Language. It supports large-number input, validation, reversed-array storage, addition, subtraction, multiplication, output formatting, and basic analysis. The project follows the

required modular development strategy and demonstrates collaborative implementation, testing, debugging, and integration.

The Expression Mode extension (`expression.asm`) significantly increases the capability of the calculator by allowing users to type full mathematical expressions. It correctly applies BODMAS precedence, handles all three bracket types, supports four number systems, and provides specific error messages for each type of invalid input. As shown in Section 6, its thirteen procedures were divided into balanced groups of three to four across all four members, so that the expression evaluator — like the rest of the project — reflects an equal collaborative contribution from the whole team. All features have been tested and verified, as documented in Section 12.